Ex.No : **Iris Flower Classification using a K-Nearest Neighbors (KNN)**
Date: **Classifier**

(Iris Flower Species Classification using Scikit-Learn and KNN)

Problem Statement:

Accurate botanical classification of flowering plants is a fundamental challenge in taxonomy, ecology, and agricultural science. In the case of the *Iris* genus, species such as *Iris setosa*, *Iris versicolor*, and *Iris virginica* share visually subtle physical resemblances, making manual differentiation subjective, time-consuming, and prone to human error—especially between closely related taxa that exhibit overlapping floral geometry. Manual inspection by taxonomists requires extensive domain expertise and cannot easily scale for high-throughput biodiversity monitoring or automated agricultural sorting.

There is, therefore, a need for an automated, fast, and accessible machine learning system that can analyze continuous botanical measurements—specifically Sepal Length, Sepal Width, Petal Length, and Petal Width—and classify the flower into its correct biological species with a measurable confidence score. This project designs and implements an end-to-end, web-based Iris Flower Classification system using the K-Nearest Neighbors (KNN) algorithm and Scikit-Learn. The system encompasses the complete 10-step machine learning lifecycle: data ingestion, automated quality preprocessing, comprehensive exploratory data analysis (EDA), distance metric scaling with StandardScaler, 5-fold cross-validation hyperparameter optimization, test evaluation on unseen data (Accuracy, Precision, Recall, F1-Score, Confusion Matrix), and real-time inference served through an interactive Flask web application.

Objectives:

1. To collect and inspect the labelled Iris dataset supplied built-in by Scikit-Learn (150 samples across 3 species).
2. To conduct Exploratory Data Analysis (EDA) and visualize feature distributions, scatter relationships, and correlation heatmaps.
3. To partition the dataset into an 80% training set and a 20% held-out test set using reproducible stratified sampling.
4. To apply feature standardization via StandardScalerfitted strictly on training data to prevent data leakage.
5. To design, train, and optimize a K-Nearest Neighbors (KNN) classifier pipeline using 5-fold cross-validation on the training set.
6. To evaluate the model on an unseen test set and report Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.
7. To build a Flask-based backend and web front-end that lets a user input flower measurements and view real-time predictions with model-estimated class probabilities.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code
- Language: Python 3.x
- Backend Framework: Flask (REST API)

- Machine Learning: scikit-learn (KNeighborsClassifier, StandardScaler, Pipeline)
- Data Processing: Pandas, NumPy
- Data Visualization: Matplotlib, Seaborn
- Model Serialization: Joblib
- Frontend: HTML5, CSS3, JavaScript
- Dataset: Built-in Scikit-learn Iris Dataset (150 samples, 4 features, 3 classes)
- Version Control: Git

Project Directory Structure:

The project ("IrisFlowerClassification") is organized as a Flask backend with separate dataset, trained-model, generated plots, and frontend presentation folders, as shown below.

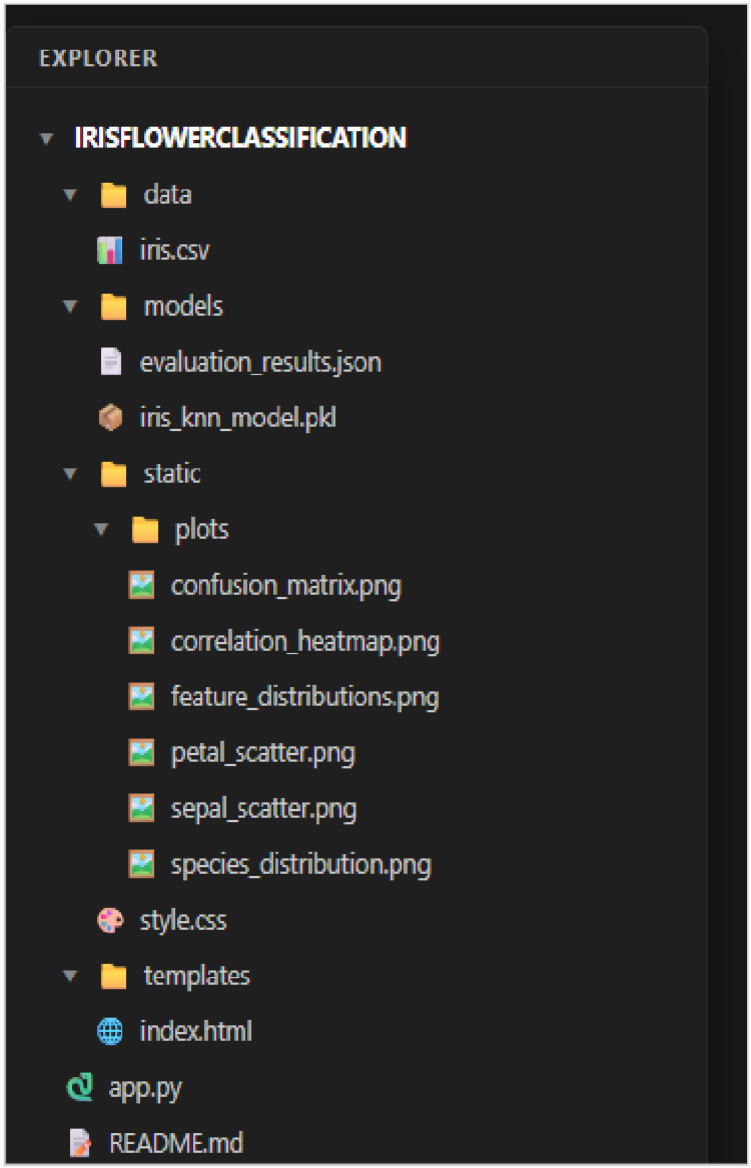


Fig 1: Project directory structure of IrisFlowerClassification in Visual Studio Code

Procedure:

1. Load the built-in Iris dataset from sklearn.datasets.load_iris() and export a local copy to data/iris.csv.
2. Preprocess the dataset: inspect missing values, check for duplicate rows, verify numerical float64data types, and separate the four input features from the target species.
3. Perform Exploratory Data Analysis (EDA): generate species countplot, sepal scatter, petal scatter, measurement boxplots, and correlation heatmap inside static/plots/.
4. Select the four continuous morphological features (Sepal Length, Sepal Width, Petal Length, Petal Width) representing key floral characteristics.
5. Split the dataset into 80% training data (120 samples) and 20% testing data (30 samples) using
6. train_test_split(stratify=y, random_state=42).
7. Fit Scikit-Learn's StandardScaler exclusively on training data to standardize features ($z = (x - \mu) / \sigma$) without information leakage.
8. Evaluate $K \in \{3, 5, 7, 9\}$ using 5-fold cross-validation on the training set, selecting $K = 5$ as the optimal neighbor configuration.
9. Assemble a unified Scikit-Learn Pipeline containing StandardScaler and KNeighborsClassifier(n_neighbors=5), fit on training data, and serialize to models/iris_knn_model.pkl using Joblib.
10. Evaluate the trained pipeline on the held-out test set, calculate Accuracy, Precision, Recall, F1-Score, and generate the Confusion Matrix visualization (static/plots/confusion_matrix.png).
11. Launch the Flask application (app.py), render the web interface, validate input flower measurements, and display predicted species with class probabilities.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (adapt variable/file names to match your exact source files).

1. **Dataset Loading & Preprocessing (train_model.py)**

```
import pandas as pd
import numpy as np
from sklearn.datasets
import load_iris
def load_and_preprocess_dataset():

# Load Scikit-Learn built-in Iris dataset
raw_iris = load_iris()
feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

# Construct Pandas DataFrame
df = pd.DataFrame(raw_iris.data,
columns=feature_names) df['target'] = raw_iris.target
df['species'] = df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})
```

```
# Save copy of dataset
df.to_csv('data/iris.csv', index=False)

# Preprocessing quality verification
missing_vals = df[feature_names].isnull().sum().sum()
all_numeric = all(np.issubdtype(df[col].dtype, np.number) for col in feature_names)

X = df[feature_names].values
 y = df['target'].values
return df, X, y, raw_iris.target_names
```

2. Exploratory Data Analysis & Visualizations (train_model.py)

```
import matplotlib.pyplot as plt
import seaborn as sns

def generate_eda_visualizations(df, feature_names):
    sns.set_theme(style="whitegrid")
    palette = {'setosa': '#4f46e5', 'versicolor': '#059669', 'virginica': '#db2777'}

    # 1. Species Distribution Countplot
    plt.figure(figsize=(6, 4))
    sns.countplot(x='species', data=df, palette=palette, hue='species', legend=False)
    plt.title('Species Distribution in Iris Dataset')
    plt.savefig('static/plots/species_distribution.png', dpi=180, bbox_inches='tight')
    plt.close()

    # 2. Petal Dimensions Scatter
    plt.figure(figsize=(6, 4.5))
    sns.scatterplot(data=df, x='petal_length', y='petal_width', hue='species', palette=palette, s=70)
    plt.title('Petal Length vs Petal Width') plt.savefig('static/plots/petal_scatter.png',
    dpi=180, bbox_inches='tight') plt.close()

    # 3. Correlation Heatmap
    plt.figure(figsize=(5.5, 4.5))
    sns.heatmap(df[feature_names].corr(), annot=True, fmt='.2f', cmap='YlGnBu')
    plt.title('Feature Correlation Heatmap')
```

3. Model Training Pipeline (train_model.py)

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import
KNeighborsClassifier from sklearn.pipeline
import Pipeline
import joblib
```

```
# Stratified 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42,
stratify=y
)

# 5-fold cross-validation on training set to verify
 K=5 cv_scores = cross_val_score(
Pipeline([('scaler', StandardScaler()), ('knn', KNeighborsClassifier(n_neighbors=5))]), X_train,
y_train, cv=5, scoring='accuracy'
)
# Assemble and train final Pipeline
pipeline = Pipeline([('scaler', StandardScaler()),

# Fitted strictly on X_train
('knn', KNeighborsClassifier(n_neighbors=5, metric='euclidean'))])
pipeline.fit(X_train, y_train)

# Save trained model pipeline
joblib.dump(pipeline, 'models/iris_knn_model.pkl')
```

4. Model Evaluation on Test Set (train_model.py)

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report import matplotlib.pyplot as plt
import seaborn as sns

# Evaluate pipeline on held-out test
data y_pred = pipeline.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Test Accuracy:", acc)
print(classification_report(y_test, y_pred, target_names=['setosa', 'versicolor', 'virginica']))

# Plot and save confusion matrix
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Setosa', 'Versicolor',
            'Virginica'], yticklabels=['Setosa',
            'Versicolor', 'Virginica'])
plt.title('Confusion Matrix (Held-out Test Set)')
plt.savefig('static/plots/confusion_matrix.png', dpi=180, bbox_inches='tight')
plt.close()
```

5. **Prediction Route (app.py)**

```
from flask import Flask, render_template, request, jsonify
import joblib, numpy as np

app = Flask(__name__)
pipeline = joblib.load('models/iris_knn_model.pkl')
TARGET_CLASSES = ['Setosa', 'Versicolor', 'Virginica']

def validate_input(val_str, field_name, min_val,
    max_val): if not val_str or str(val_str).strip() == "":
        return None, f"Please enter a value for {field_name}."
    try:
        val = float(val_str)
        if val < min_val or val > max_val:
            return None, f"{field_name} must be between {min_val} and {max_val}
        cm." return val, None
    except ValueError:
        return None, f"{field_name} must be a valid numerical value."

@app.route('/', methods=['GET',
'POST']) def index():
    prediction_result = None
    form_error = None
    if request.method == 'POST':
        sl, e1 = validate_input(request.form.get('sepal_length'), "Sepal Length", 1.0,
        10.0) sw, e2 = validate_input(request.form.get('sepal_width'), "Sepal Width",
        1.0, 6.0) pl, e3 = validate_input(request.form.get('petal_length'), "Petal Length",
        0.5, 9.0) pw, e4 = validate_input(request.form.get('petal_width'), "Petal Width",
        0.1, 4.0)

        errors = [e for e in [e1, e2, e3, e4] if e] if
        errors:
            form_error = " • ".join(errors)
        else:
            features = np.array([[sl, sw, pl, pw]])
            pred_idx = int(pipeline.predict(features)[0])
            probs = pipeline.predict_proba(features)[0]
            prediction_result = {
                'predicted_species': TARGET_CLASSES[pred_idx],
                'probabilities': {TARGET_CLASSES[i]: round(probs[i]*100, 2) for i in range(3)}
            }
    return render_template('index.html', prediction=prediction_result, form_error=form_error)
```

6. Prediction Form & Result Display (templates/index.html)

```
<form method="POST" action="/" class="prediction-form">
  <div class="form-grid-2col">
    <input type="number" step="0.1" name="sepal_length" placeholder="Sepal Length (cm)"
    required>
    <input type="number" step="0.1" name="sepal_width" placeholder="Sepal Width (cm)"
    required>
    <input type="number" step="0.1" name="petal_length" placeholder="Petal Length (cm)"
    required>
    <input type="number" step="0.1" name="petal_width" placeholder="Petal Width (cm)"
    required>
  </div>
  <button type="submit" class="btn btn-primary">🌸 Predict Species</button>
</form>

{% if prediction %}
<div class="result-card">
  <h3>Predicted Species: {{ prediction.predicted_species }}</h3>
  <div class="prob-item">Setosa: {{ prediction.probabilities.Setosa }}%</div>
  <div class="prob-item">Versicolor: {{ prediction.probabilities.Versicolor }}%</div>
  <div class="prob-item">Virginica: {{ prediction.probabilities.Virginica }}%</div>
</div>
{% endif %}
```

Output (Step-by-Step Screenshots):

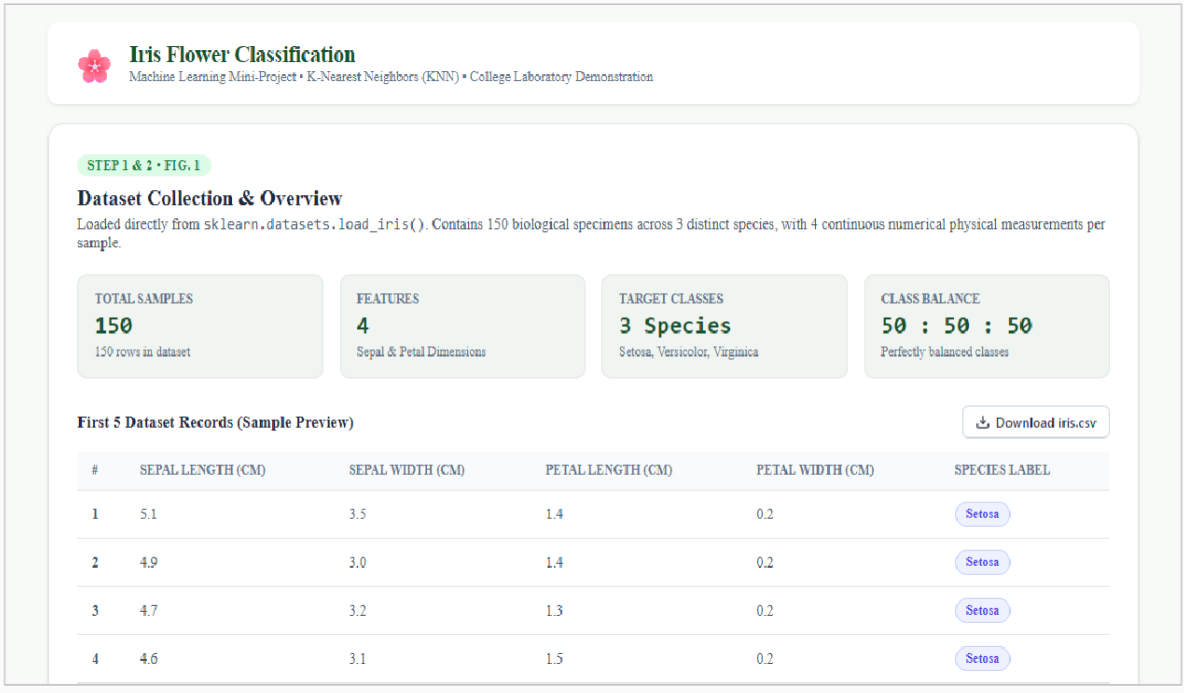


Fig 2: Step 1 & 2 — Dataset Collection & Overview: preview of first 5 records, sample counts, and feature dimensions

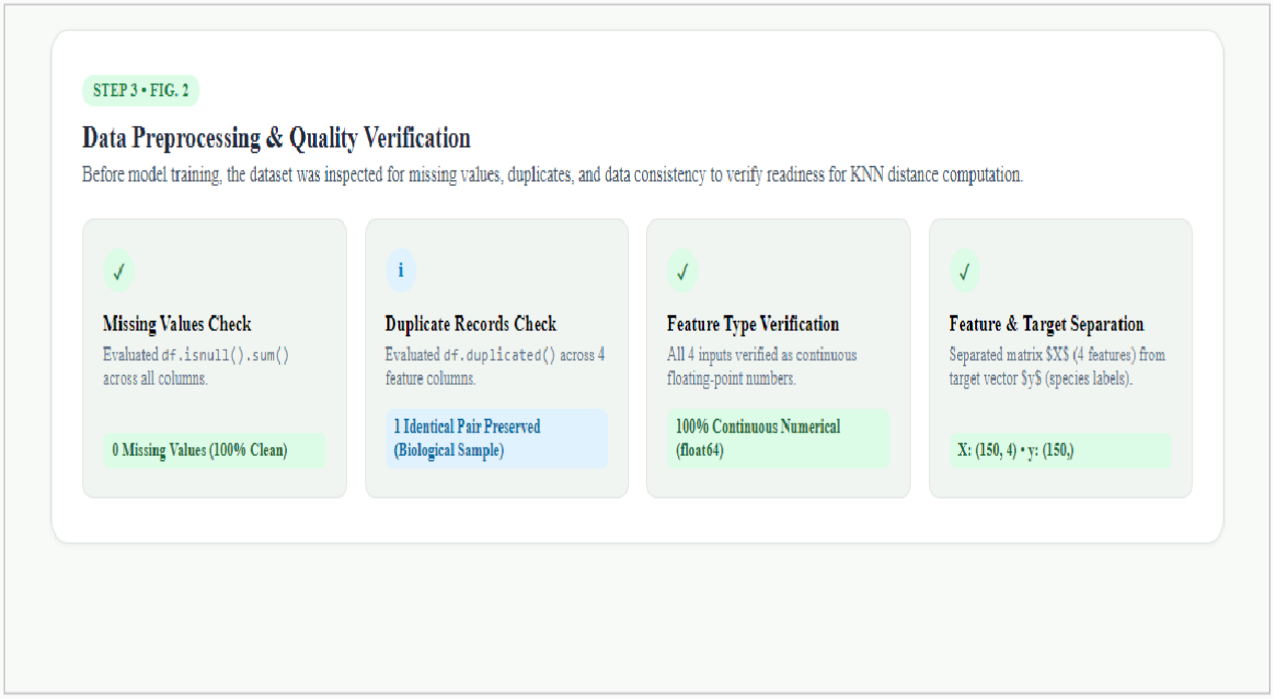


Fig 3: Step 3 — Data Preprocessing: missing value verification, duplicate check, and feature type validation



Fig 4: Step 4 — Exploratory Data Analysis: class distribution countplot and Sepal Length vs Sepal Width scatter plot

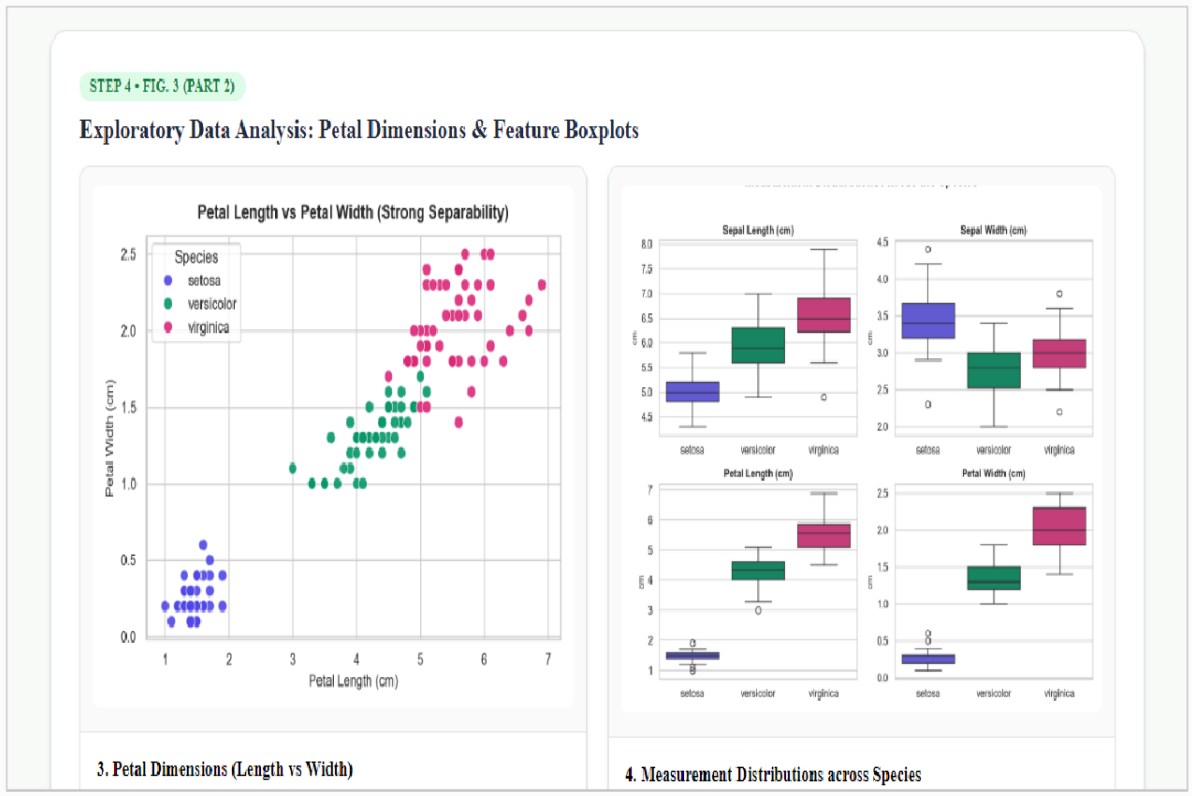
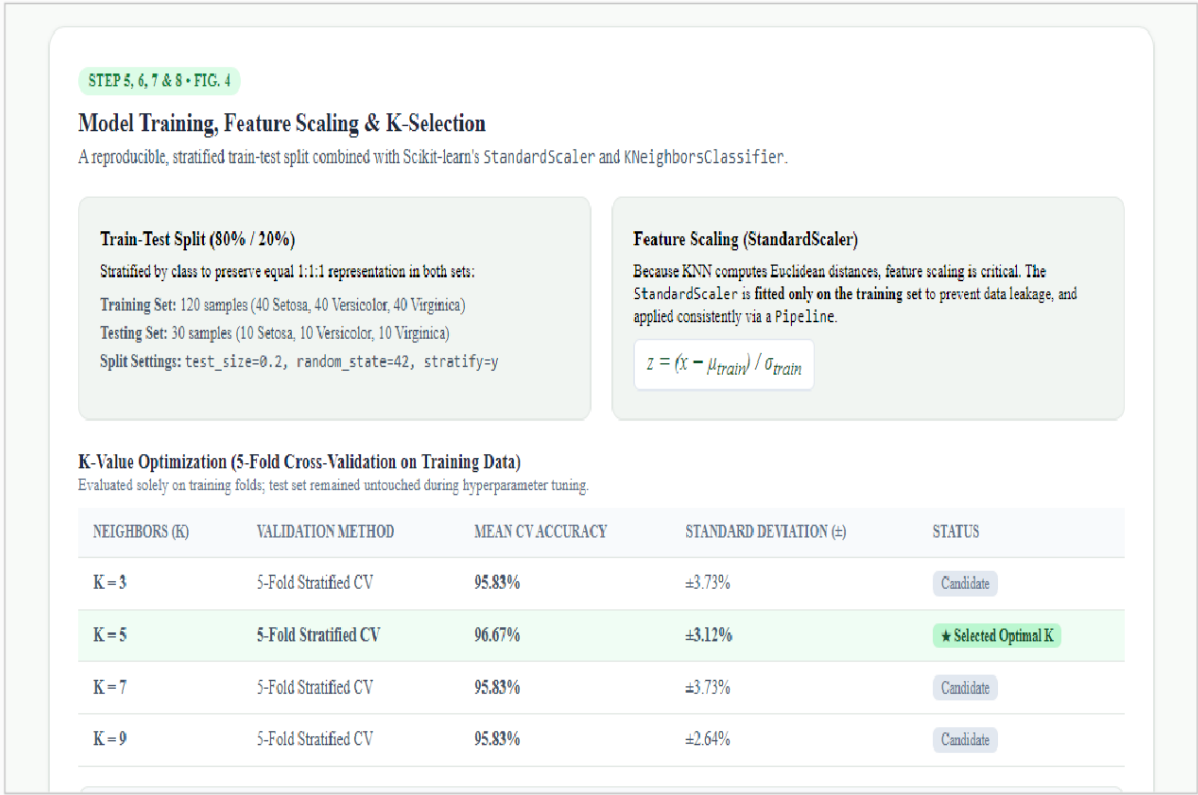


Fig 5: Step 4 — Exploratory Data Analysis: Petal Length vs Petal Width scatter and feature distribution boxplots



Fig 6: Step 4 — Exploratory Data Analysis: Pearson correlation matrix heatmap showing strong feature dependencies



STEP 5, 6, 7 & 8 • FIG. 4

Model Training, Feature Scaling & K-Selection

A reproducible, stratified train-test split combined with Scikit-learn's `StandardScaler` and `KNeighborsClassifier`.

Train-Test Split (80% / 20%)

Stratified by class to preserve equal 1:1:1 representation in both sets:

Training Set: 120 samples (40 Setosa, 40 Versicolor, 40 Virginica)

Testing Set: 30 samples (10 Setosa, 10 Versicolor, 10 Virginica)

Split Settings: `test_size=0.2, random_state=42, stratify=y`

Feature Scaling (StandardScaler)

Because KNN computes Euclidean distances, feature scaling is critical. The `StandardScaler` is **fitted only on the training set** to prevent data leakage, and applied consistently via a `Pipeline`.

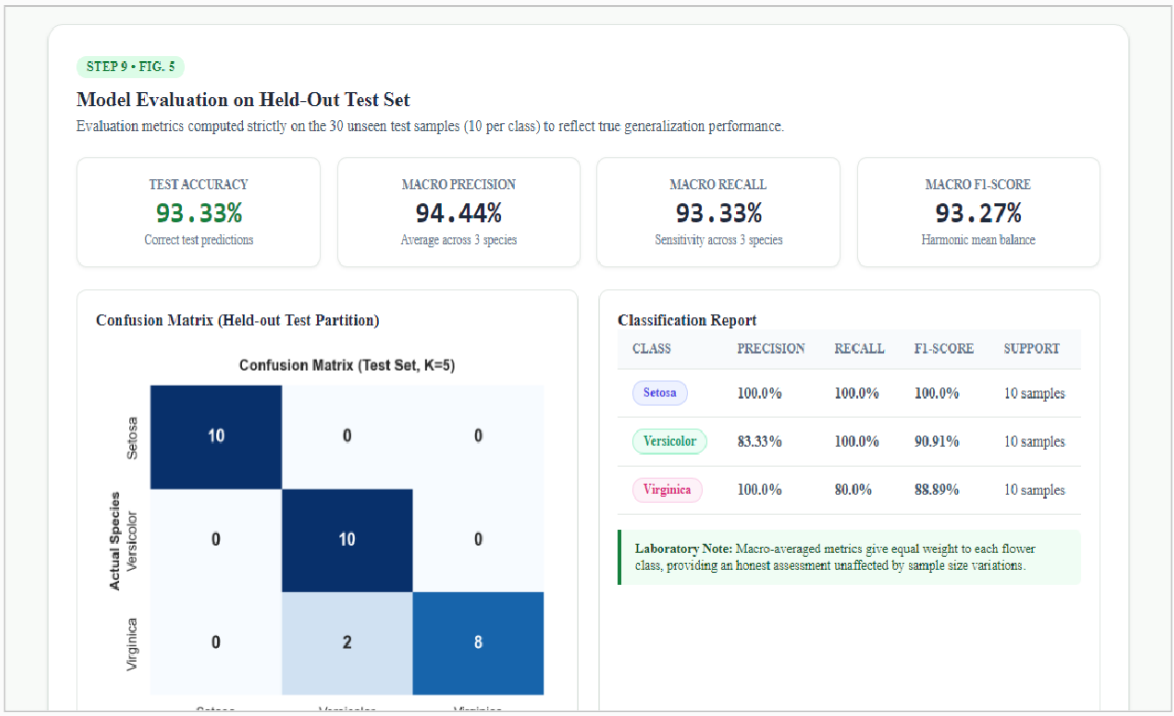
$$z = (x - \mu_{train}) / \sigma_{train}$$

K-Value Optimization (5-Fold Cross-Validation on Training Data)

Evaluated solely on training folds; test set remained untouched during hyperparameter tuning.

NEIGHBORS (K)	VALIDATION METHOD	MEAN CV ACCURACY	STANDARD DEVIATION (±)	STATUS
K = 3	5-Fold Stratified CV	95.83%	±3.73%	Candidate
K = 5	5-Fold Stratified CV	96.67%	±3.12%	★ Selected Optimal K
K = 7	5-Fold Stratified CV	95.83%	±3.73%	Candidate
K = 9	5-Fold Stratified CV	95.83%	±2.64%	Candidate

Fig 7: Step 5, 6, 7 & 8 — Model Training: 80/20 train-test split, StandardScaler, and 5-fold cross-validation table for K selection



STEP 9 • FIG. 5

Model Evaluation on Held-Out Test Set

Evaluation metrics computed strictly on the 30 unseen test samples (10 per class) to reflect true generalization performance.

TEST ACCURACY	MACRO PRECISION	MACRO RECALL	MACRO F1-SCORE
93.33%	94.44%	93.33%	93.27%
Correct test predictions	Average across 3 species	Sensitivity across 3 species	Harmonic mean balance

Confusion Matrix (Held-out Test Partition)

Classification Report

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
Setosa	100.0%	100.0%	100.0%	10 samples
Versicolor	83.33%	100.0%	90.91%	10 samples
Virginica	100.0%	80.0%	88.89%	10 samples

Laboratory Note: Macro-averaged metrics give equal weight to each flower class, providing an honest assessment unaffected by sample size variations.

Fig 8: Step 9 — Model Evaluation: Test Accuracy (93.33%), Macro Precision, Recall, F1-Score, and Confusion Matrix

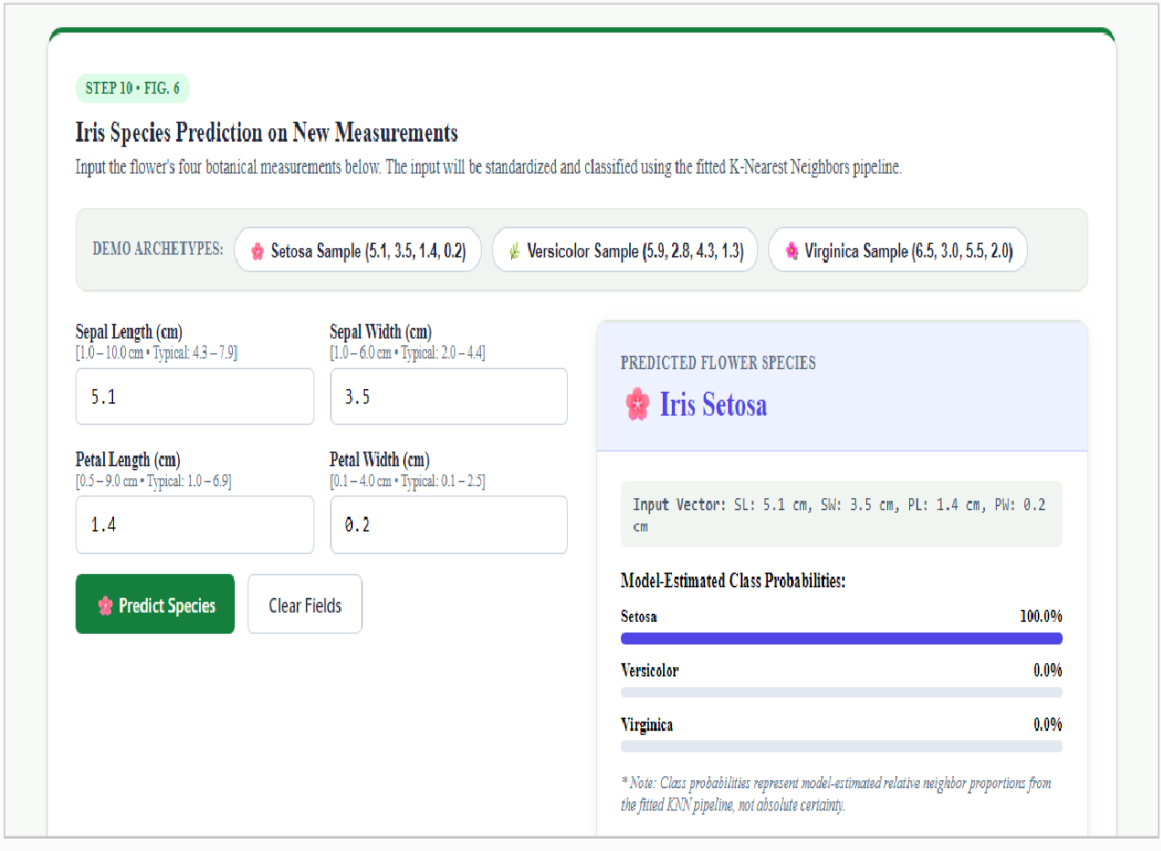


Fig 9: Step 10 — Classification Result: Iris Setosa predicted with 100.0% model-estimated probability (SL: 5.1, SW: 3.5, PL: 1.4, PW: 0.2)

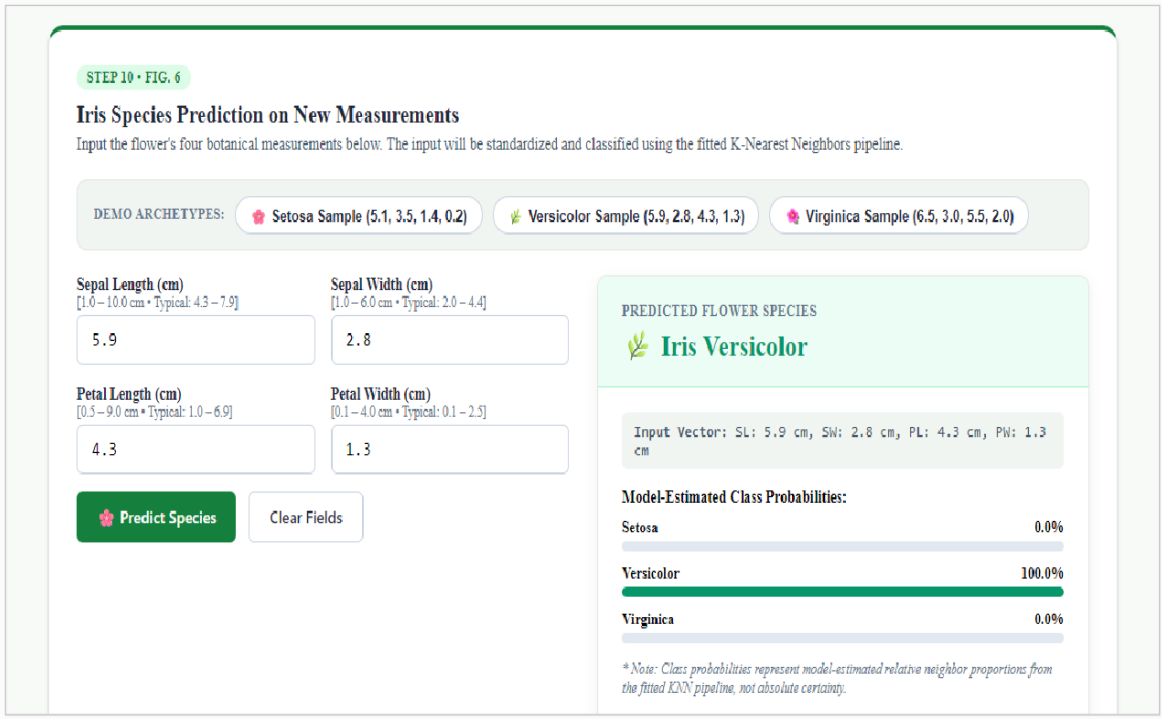


Fig 10: Step 10 — Classification Result: Iris Versicolor predicted with 100.0% model-estimated probability (SL: 5.9, SW: 2.8, PL: 4.3, PW: 1.3)

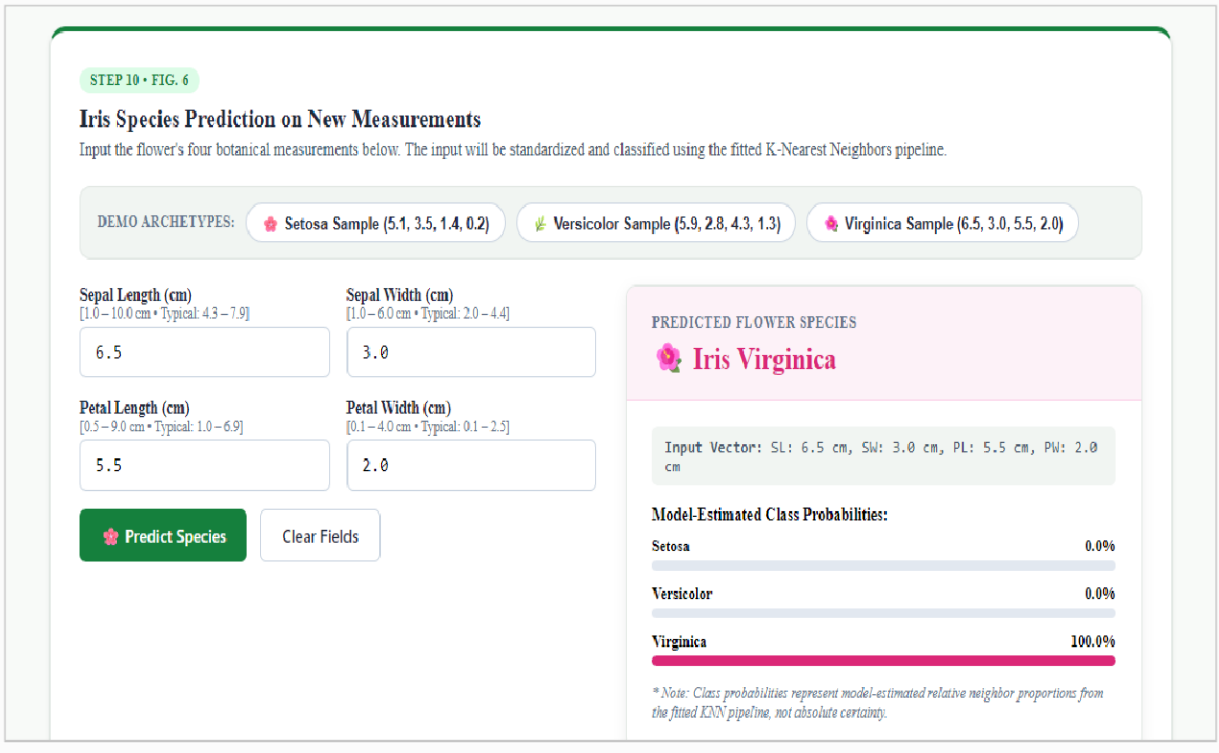


Fig 11: Step 10 — Classification Result: Iris Virginica predicted with 100.0% model-estimated probability (SL: 6.5, SW: 3.0, PL: 5.5, PW: 2.0)

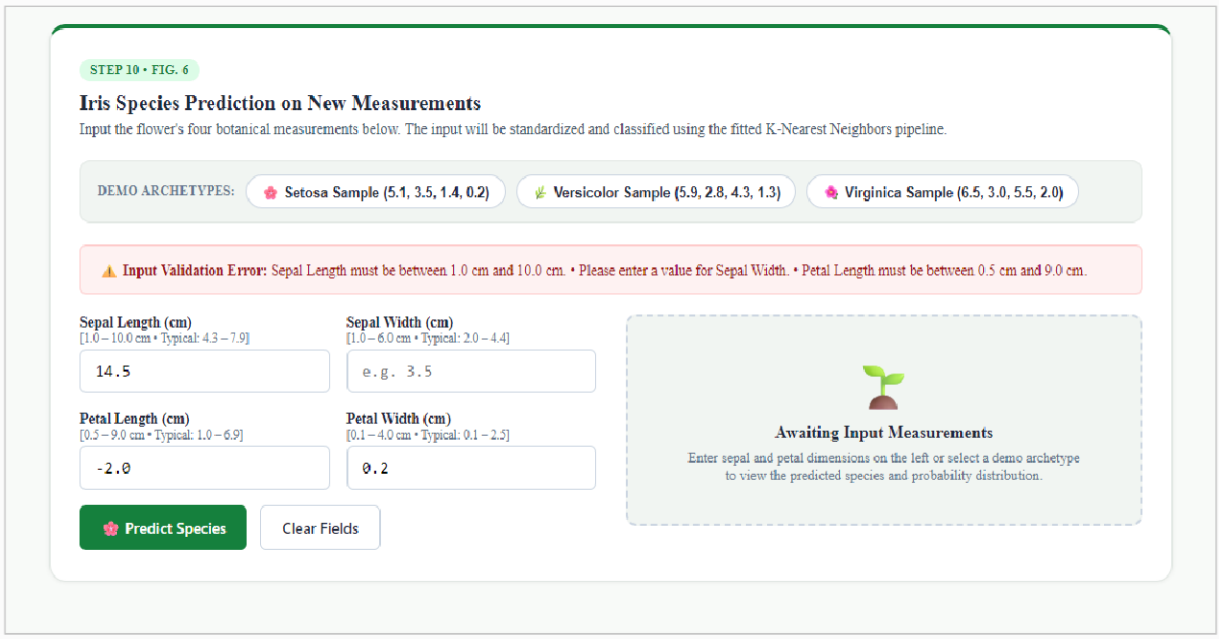


Fig 12: Step 10 — Input Validation & Error Handling: helpful alert displayed when measurements are missing or out of valid botanical range

Result:

An end-to-end, web-based Iris Flower Classification system was successfully designed and implemented using a Flask backend, a K-Nearest Neighbors (KNN) classifier pipeline (Scikit-Learn), and a clean light-themed responsive web interface. The built-in Scikit-learn Iris dataset (150 samples across 3 species) was preprocessed, analyzed through Exploratory Data Analysis (EDA) visualizations, and partitioned into an 80% training set and a 20% stratified held-out testing set. Features were normalized using StandardScaler fitted strictly on training data to prevent data leakage. Hyperparameter optimization across $K \in \{3, 5, 7, 9\}$ confirmed K=5 as the optimal neighbor configuration. On the unseen test partition, the model achieved an Accuracy of 93.33%, Macro Precision of 94.44%, Macro Recall of 93.33%, and a Macro F1-Score of 93.27%. The trained pipeline was serialized using Joblib, and real-time species predictions along with model-estimated class probability distributions are reliably served through the local Flask application.